Proyecto 1 — Entrega 2

Integración continua de un microservicio sobre AWS CodeBuild y AWS CodePipeline

Programa: Maestría en Inteligencia Artificial (MAIA)

Universidad de los Andes

Autor: Jesús Daniel Lorduy — j.lorduy@uniandes.edu.co

Fecha: 31 de octubre de 2026

Enlaces de la entrega

Recurso	Enlace
Repositorio GitHub	https://github.com/LorduyANDES/blacklist-microservice
Aplicación desplegada	http://blacklist-env.eba-2qryjvm9.us-east-2.elasticbeanstalk.com
Documentación publicada de Postman	https://documenter.getpostman.com/view/59043927/2sBYHSy11e
Video de sustentación	`[COMPLETAR: URL del video, máximo 10 minutos]`

> *La URL de la aplicación cambió respecto a la entrega 1 porque el entorno de Elastic Beanstalk se recreó para esta entrega. El environment de Postman debe actualizarse con la URL de la tabla anterior.*

---

1. Alcance de esta entrega

Esta entrega construye un **pipeline de integración continua** sobre el microservicio de lista negra global desarrollado en la entrega 1. Cada vez que se registra un commit en la rama `main` del repositorio, AWS CodePipeline arranca automáticamente, AWS CodeBuild instala las dependencias, ejecuta las pruebas unitarias y, **solo si todas pasan**, empaqueta el artefacto desplegable y lo publica en S3.

El alcance se detiene deliberadamente ahí. **No hay entrega continua:** el artefacto se genera y se almacena, pero ningún paso del pipeline lo despliega en Elastic Beanstalk. El despliegue sigue siendo manual, como exige el enunciado de esta entrega. Esa decisión está escrita de forma explícita en el propio `buildspec.yml` y aparece en el log de cada construcción exitosa:

```
Artefacto generado - NO se despliega, esta entrega cubre solo CI
```

La aplicación permanece en ejecución sobre Elastic Beanstalk y responde a las peticiones de Postman durante toda la entrega.

---

2. Arquitectura del pipeline



Recursos creados:

Recurso	Nombre	Región
Proyecto de CodeBuild	`blacklist-microservice-build`	us-east-2
Pipeline	`blacklist-microservice-ci` (tipo V2)	us-east-2
Conexión a GitHub	`blacklist-github` (AWS CodeConnections)	us-east-2
Rol de CodeBuild	`blacklist-codebuild-role`	—
Rol de CodePipeline	`blacklist-pipeline-role`	—
Grupo de informes	`blacklist-microservice-build-pruebas-unitarias`	us-east-2

---

3. Pruebas unitarias de cada endpoint

El proyecto tiene **25 pruebas unitarias** en `tests/test_blacklists.py`, escritas con `pytest` y el cliente de pruebas de Flask. Cubren los tres endpoints del API.

3.1 Estrategia de aislamiento

Las pruebas corren contra **SQLite en un archivo temporal**, no contra el RDS. Eso tiene dos consecuencias deliberadas:

1. La construcción en CodeBuild **no necesita levantar un motor de base de datos** ni credenciales de producción, de modo que el pipeline no toca datos reales ni expone secretos.
2. El modelo usa únicamente tipos portables (`String`, `DateTime`, `Integer`), por lo que el comportamiento verificado es el mismo que sobre PostgreSQL.

Cada prueba recibe una base limpia mediante *fixtures* definidas en `conftest.py`.

3.2 Cobertura por endpoint

`GET /` — health check (1 prueba)

Prueba	Qué verifica
`test_health_check_responde_ok_sin_token`	Responde 200 sin exigir token, porque lo consume el health check del balanceador

Autorización, transversal a los endpoints de negocio (4 pruebas)

Prueba	Qué verifica
`test_post_sin_encabezado_de_autorizacion_es_401`	Falta el encabezado `Authorization`
`test_post_con_token_incorrecto_es_401`	Token que no corresponde
`test_post_con_esquema_no_bearer_es_401`	Esquema distinto de `Bearer`
`test_get_sin_token_es_401`	La consulta también exige token

`POST /blacklists` (15 pruebas)

Prueba	Qué verifica
`test_post_valido_crea_la_entrada`	Caso feliz: responde **201** con `created: true`
`test_post_registra_ip_y_fecha_internamente`	La IP y la fecha las registra el servicio, no el cliente
`test_post_usa_la_ip_del_cliente_detras_del_balanceador`	Con `X-Forwarded-For` de dos saltos se guarda la IP del cliente, no la del balanceador
`test_no_se_puede_falsificar_la_ip_con_un_solo_salto`	Un cliente no puede suplantar su IP inyectando un solo salto
`test_blocked_reason_es_opcional`	El motivo puede omitirse
`test_email_duplicado_es_409`	Email ya listado responde 409 con `created: false`
`test_email_duplicado_con_otra_capitalizacion_es_409`	La deduplicación es insensible a mayúsculas
`test_email_invalido_es_400`	Formato de email inválido
`test_email_faltante_es_400`	Campo obligatorio ausente
`test_app_uuid_faltante_es_400`	Campo obligatorio ausente
`test_app_uuid_con_formato_invalido_es_400`	UUID mal formado
`test_blocked_reason_de_mas_de_255_caracteres_es_400`	Límite superior del motivo
`test_blocked_reason_de_exactamente_255_caracteres_es_valido`	Frontera exacta del límite
`test_campo_desconocido_es_400`	Un cuerpo con campos de más se rechaza (`unknown = RAISE`)
`test_cuerpo_que_no_es_json_es_400`	Cuerpo que no es JSON

`GET /blacklists/<email>` (4 pruebas)

Prueba	Qué verifica
`test_get_email_en_lista_negra_retorna_true_y_motivo`	`blacklisted: true` y el motivo registrado
`test_get_email_no_listado_retorna_false_y_motivo_nulo`	Un email no listado responde **200 con `false`**, no 404
`test_get_es_insensible_a_la_capitalizacion`	La consulta normaliza mayúsculas
`test_get_de_email_sin_motivo_retorna_motivo_nulo`	Motivo nulo cuando no se registró

Comportamiento general (1 prueba)

Prueba	Qué verifica
`test_ruta_inexistente_responde_json`	Los errores salen en JSON, no en HTML

3.3 Publicación de los resultados en CodeBuild

El `buildspec.yml` ejecuta pytest con `--junitxml`, y declara un bloque `reports` que publica ese XML como informe de pruebas. Gracias a eso la consola de CodeBuild no muestra solo un resultado global, sino **el detalle de cada caso**: nombre, conjunto, estado y duración. Un fallo indica exactamente qué prueba no pasó.

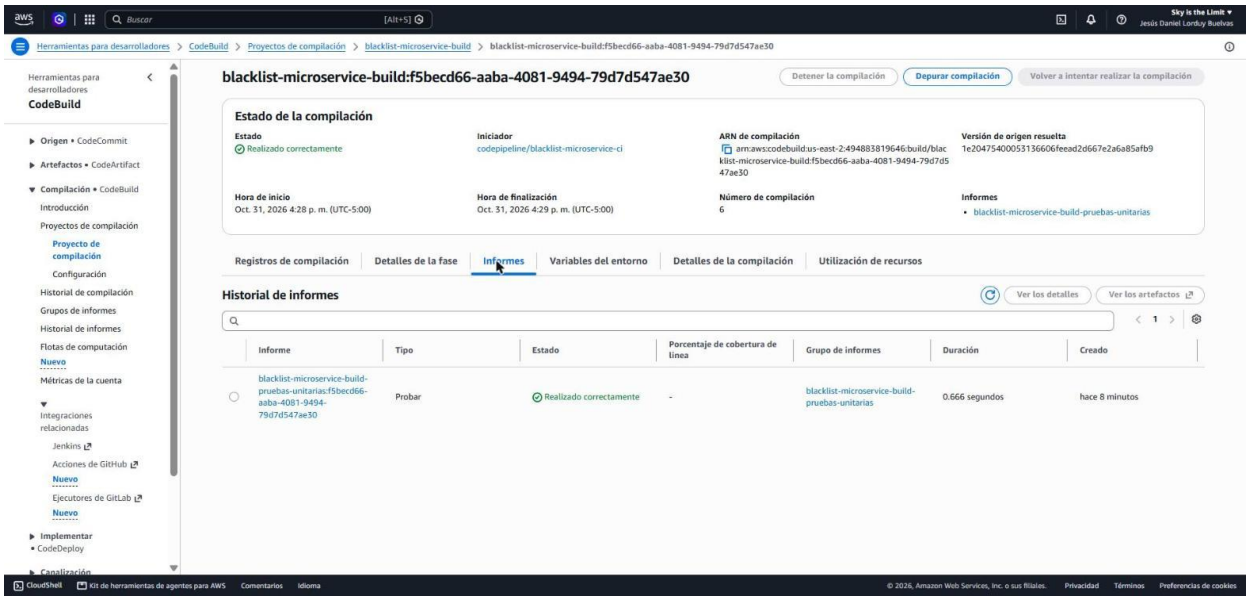


Figura 9. Pestaña *Informes* de la construcción exitosa.

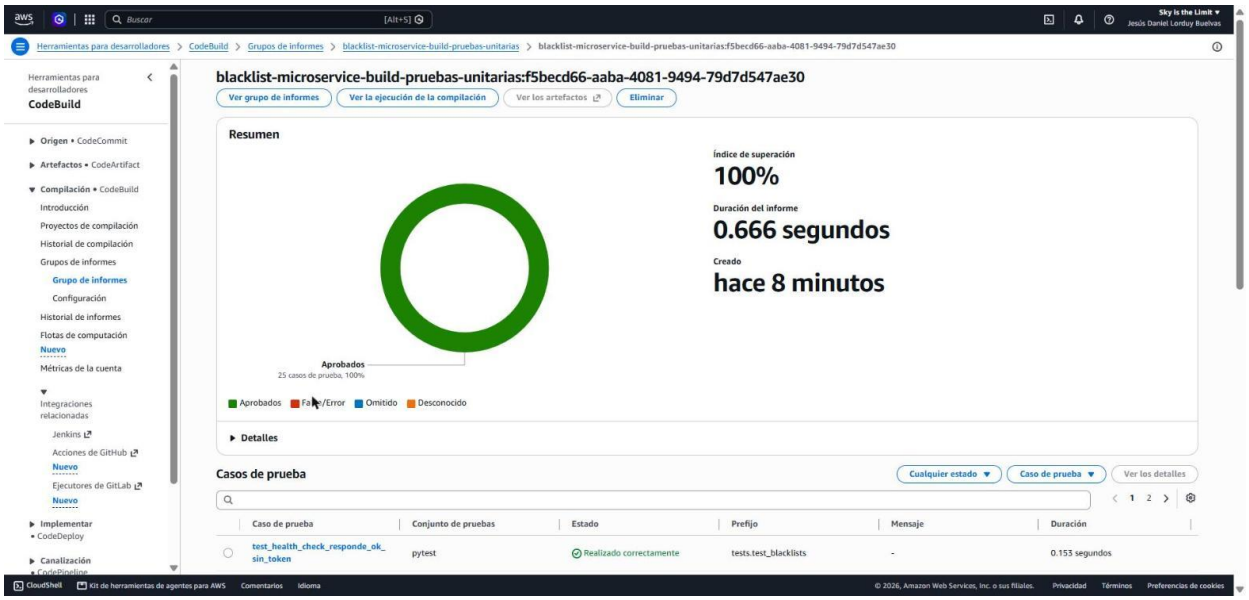


Figura 10. Informe de pruebas: índice de superación 100 %, 25 casos de prueba.

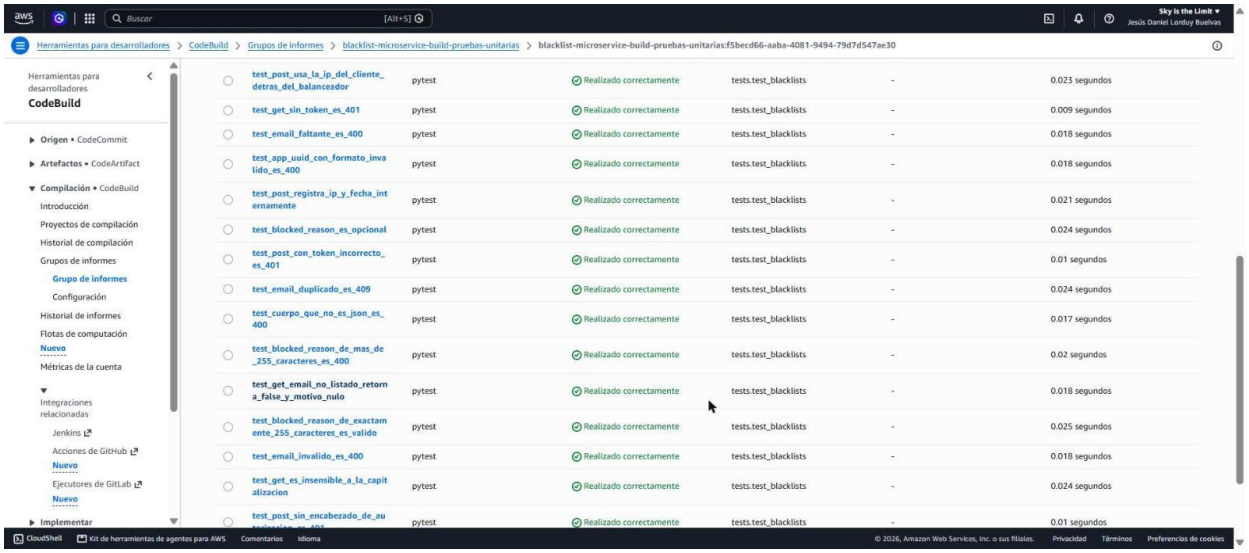


Figura 11. Detalle caso por caso.

---

4. Proyecto de AWS CodeBuild

El proyecto `blacklist-microservice-build` es el que ejecuta las pruebas y construye el artefacto.

Parámetro	Valor
Proveedor del origen	AWS CodePipeline
Imagen	`aws/codebuild/amazonlinux-x86_64-standard:5.0`
Tipo de entorno	Contenedor Linux
Tipo de cómputo	EC2 — 2 CPU virtuales, 4 GiB de memoria
Rol de servicio	`blacklist-codebuild-role`
Especificación de compilación	`buildspec.yml` (tomado del repositorio)
Tiempo de espera	20 minutos

Parámetro	Valor
Compilaciones públicas	Deshabilitado

Dos decisiones de configuración merecen explicación:

- **El origen es CodePipeline, no GitHub directamente.** CodeBuild puede conectarse por su cuenta a un repositorio, pero en ese caso tendría su propio disparador y habría dos mecanismos compitiendo. Al declarar el origen como CodePipeline, el artefacto de código fuente lo entrega la etapa *Fuente* y existe un único punto de entrada.
- **Las compilaciones públicas están deshabilitadas.** El repositorio es público, pero los logs de construcción no tienen por qué serlo.

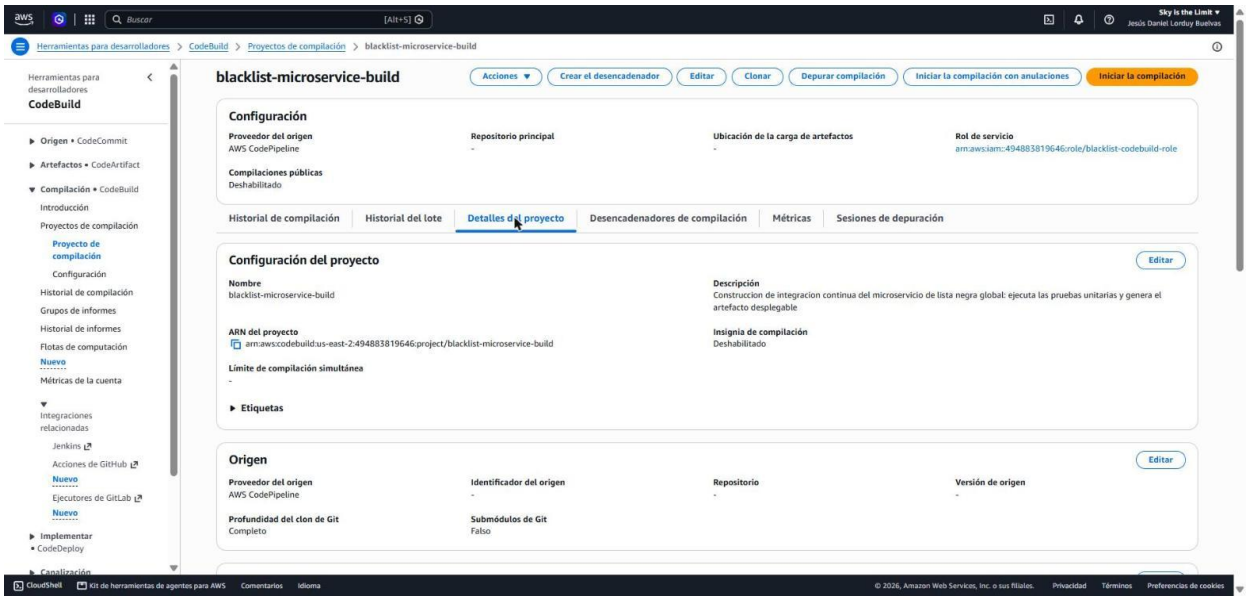


Figura 1. Configuración del proyecto de CodeBuild.

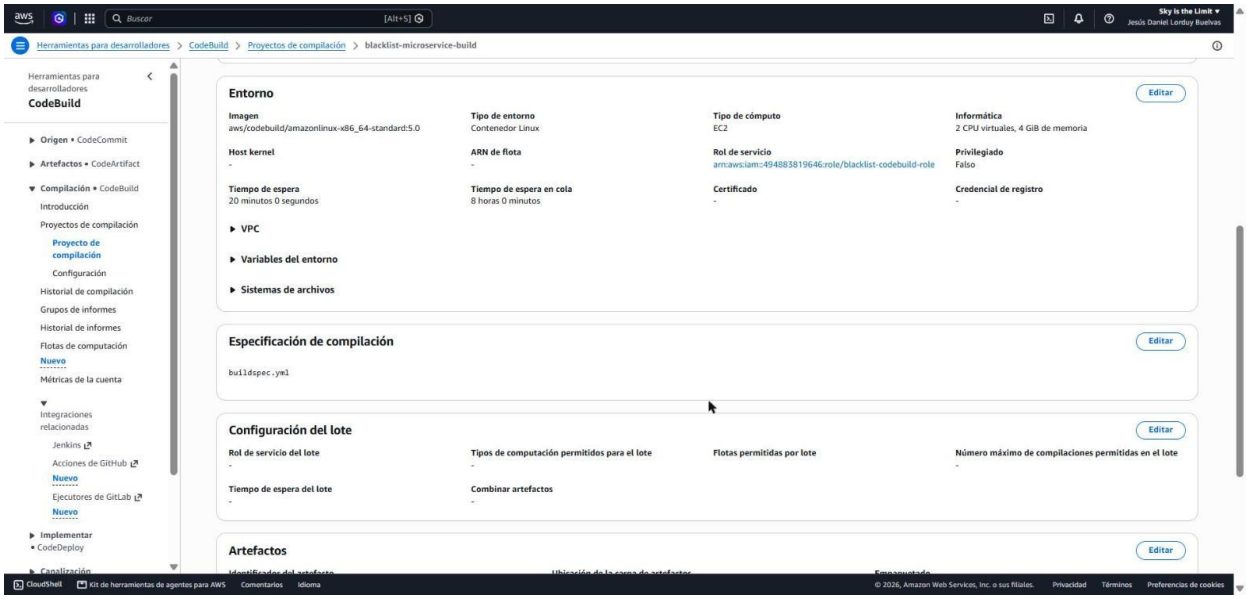


Figura 2. Entorno de ejecución y especificación de compilación.

5. El archivo `buildspec.yml` y la generación del artefacto

```
version: 0.2

env:
  variables:
    PYTHONUNBUFFERED: "1"
    PIP_DISABLE_PIP_VERSION_CHECK: "1"

phases:

  install:
    runtime-versions:
      python: 3.13
    commands:
      - echo "=== Entorno de construccion ==="
      - python --version
      - echo "Commit $CODEBUILD_RESOLVED_SOURCE_VERSION"

  pre_build:
    commands:
      - echo "=== Instalando dependencias ==="
      - python -m pip install --upgrade pip
      - pip install -r requirements-dev.txt
      - echo "=== Ejecutando las pruebas unitarias ==="
      - python -m pytest -v --junitxml=reports/pruebas-unitarias.xml

  build:
    commands:
      - echo "=== Generando el artefacto desplegable ==="
      - mkdir -p artefacto
      - cp application.py config.py models.py auth.py views.py artefacto/
      - cp requirements.txt Procfile artefacto/
      - cp -r .ebextensions artefacto/
      - cd artefacto && zip -r ../blacklist-microservice.zip . && cd ..
      - echo "=== Contenido del artefacto ==="
      - unzip -l blacklist-microservice.zip

  post_build:
    commands:
      - echo "=== Construccion finalizada ==="
      - echo "Artefacto generado - NO se despliega, esta entrega cubre solo CI"

artifacts:
  files:
    - blacklist-microservice.zip
  name: blacklist-microservice-$(date +%Y%m%d-%H%M%S)

reports:
  pruebas-unitarias:
    files:
      - reports/pruebas-unitarias.xml
    file-format: JUNITXML

cache:
  paths:
    - /root/.cache/pip/**/*
```

5.1 Por qué las pruebas van en `pre_build` y no en `build`

Es la decisión de diseño central del archivo. Las fases de CodeBuild se ejecutan en orden y **una fase que falla aborta las siguientes**. Al poner las pruebas en `pre_build`, una prueba rota impide que se llegue siquiera a crear el ZIP: el pipeline **nunca empaqueta código que no pasa sus pruebas**. Eso es precisamente lo que se espera de un pipeline de integración continua, y la sección 7.2 lo demuestra con evidencia.

5.2 Qué contiene el artefacto

El ZIP reproduce exactamente lo que espera la plataforma de Elastic Beanstalk, y nada más. Se excluyen las pruebas, el entorno virtual, la documentación y las capturas, porque ninguno de esos archivos corre en producción.

```
3088  application.py
1646  config.py
2581  models.py
1655  auth.py
3610  views.py
 236  requirements.txt
  95  Procfile
   0  .ebextensions/
1002  .ebextensions/01_packages.config
---------
13913  9 files
```

El bloque `artifacts` publica ese ZIP como salida de la etapa, y CodePipeline lo almacena en el bucket de artefactos bajo el prefijo `blacklist-microservi/ArtefactoD/`. El tamaño de cada artefacto almacenado es de **6,7 KB** (el ZIP comprimido de los 13,9 KB de origen).

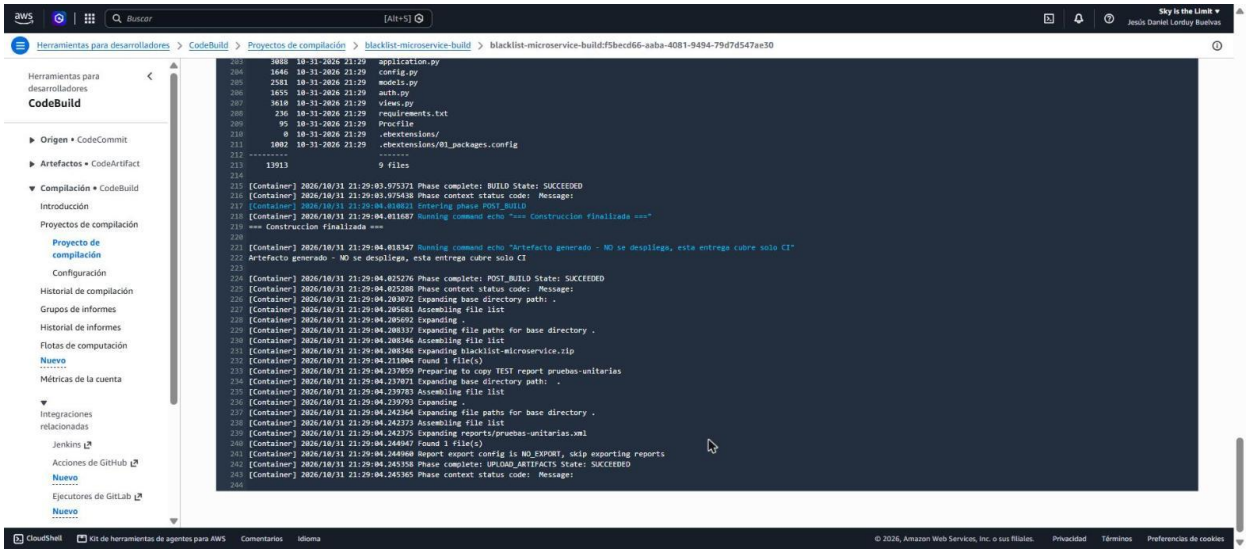


Figura 12. Log de la construcción exitosa: contenido del artefacto, fin de la fase BUILD y mensaje que confirma que no se despliega.

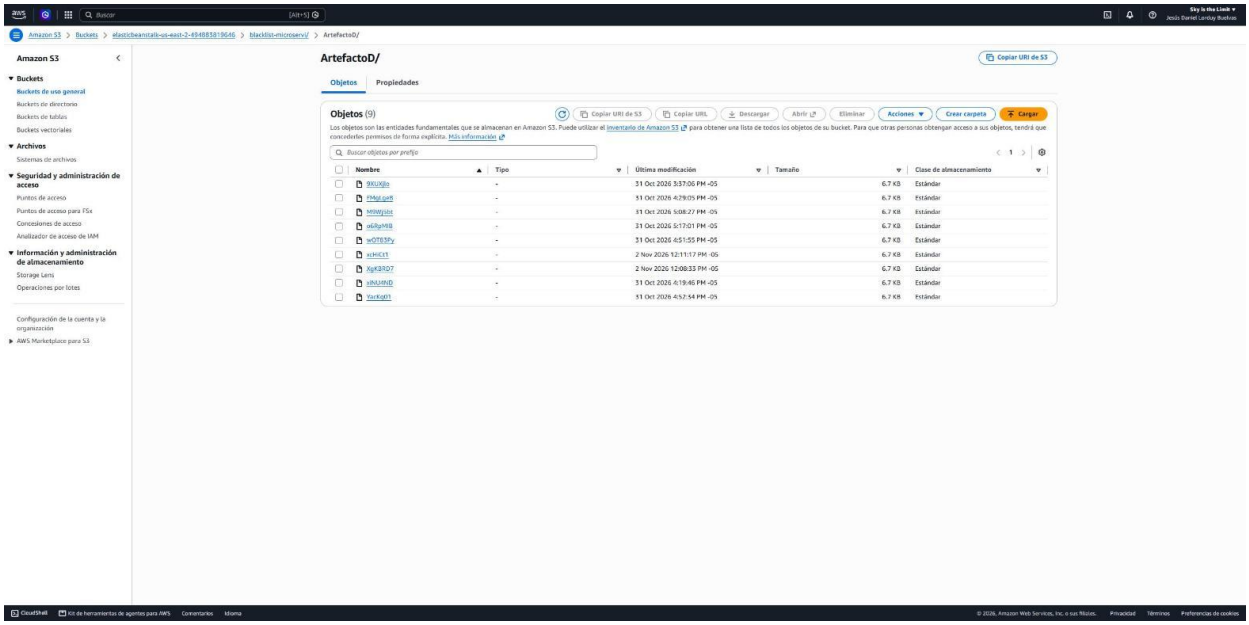


Figura 13. Artefactos almacenados en S3. Hay **uno por cada construcción exitosa, y ninguno de las fallidas**. El número crece con cada commit que pasa las pruebas, de modo que en la consola será mayor que en esta captura.

6. Pipeline y disparo automático por commit

6.1 Definición

El pipeline `blacklist-microservice-ci` es de **tipo V2** con modo de ejecución `QUEUED`. Tiene dos etapas:

```
{
  "name": "Fuente",
  "actions": [{
    "name": "GitHub",
    "actionTypeId": { "category": "Source", "owner": "AWS",
                      "provider": "CodeStarSourceConnection", "version": "1" },
    "configuration": {
      "ConnectionArn": "arn:aws:codeconnections:us-east-2:...:connection/0d950f21-...",
      "FullRepositoryId": "LorduyANDES/blacklist-microservice",
      "BranchName": "main",
      "DetectChanges": "true",
      "OutputArtifactFormat": "CODE_ZIP"
    },
    "outputArtifacts": [{ "name": "CodigoFuente" }]
  }]
}
```

y el disparador declarado a nivel de pipeline:

```
"triggers": [{
  "providerType": "CodeStarSourceConnection",
  "gitConfiguration": {
    "sourceActionName": "GitHub",
    "push": [{ "branches": { "includes": ["main"] } }]
  }
}]
```

La etapa de construcción toma `CodigoFuente` como entrada y produce `ArtefactoDesplegable` como salida.

> *El enunciado pide que el pipeline se dispare con cada commit en `master`. Este repositorio usa `main` como rama por omisión —el nombre que GitHub asigna por defecto desde 2020— y cumple exactamente el mismo papel. Tanto la acción de origen como el bloque `triggers` están declarados sobre `main`, que es la única rama protegida de hecho por el pipeline.*

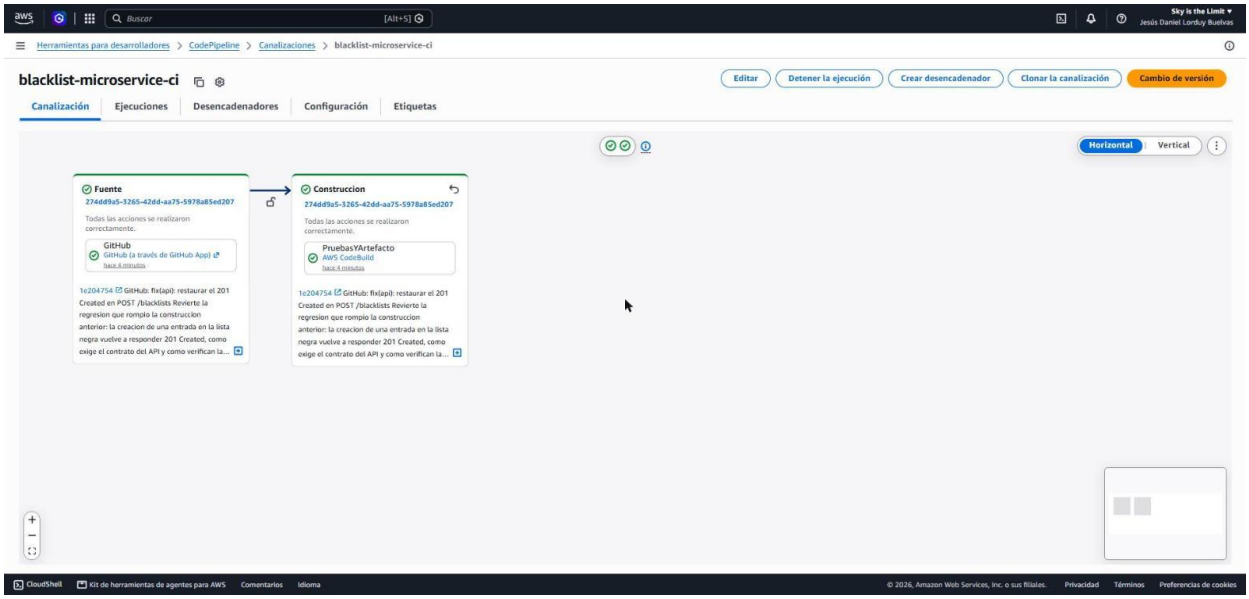


Figura 3. Vista del pipeline con las dos etapas en verde. Nótese que la consola identifica el origen como **«GitHub (a través de GitHub App)»**.

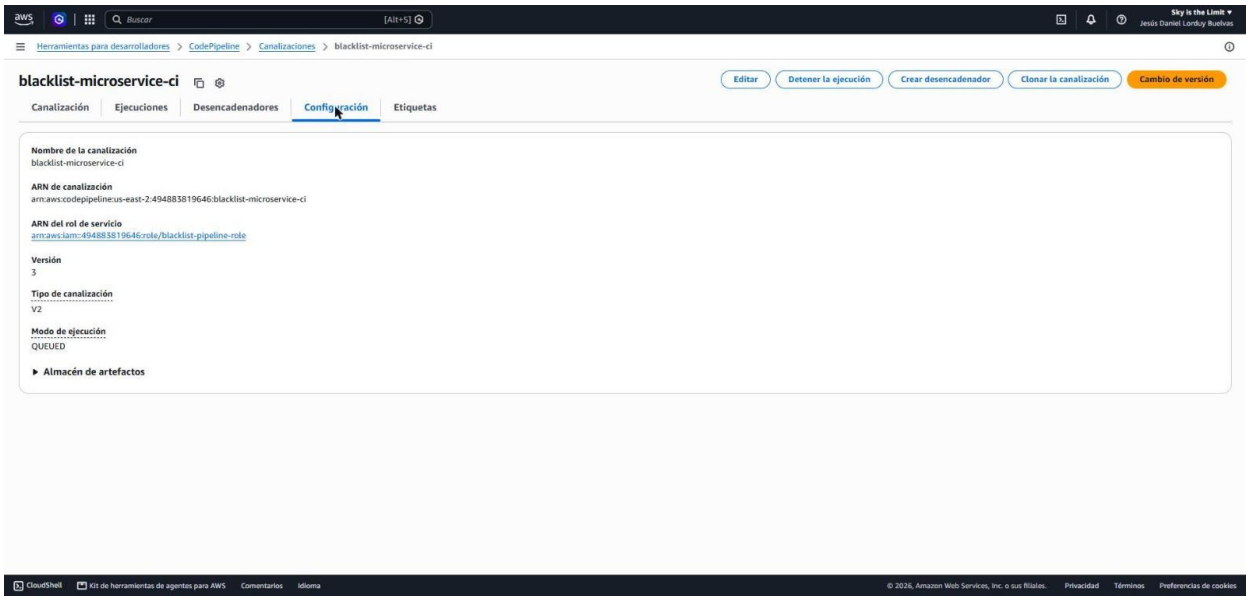


Figura 4. Configuración del pipeline: tipo V2, modo QUEUED, rol de servicio.

6.2 Evidencia del disparo automático

La columna *Desencadenador* del historial de ejecuciones es la prueba directa. Todas las ejecuciones posteriores a la instalación de la aplicación de GitHub aparecen como:

```
Commit <sha> enviado en LorduyANDES/blacklist-microservice/main
```

y su tipo de disparador en la API es `WebhookV2`, es decir, originadas por un evento de `push` entregado por GitHub, no por una acción manual. Consultado desde la CLI:

```
{
  "triggerType": "WebhookV2",
  "triggerDetail": {
    "connectionArn": "arn:aws:codeconnections:us-east-2:...:connection/0d950f21-...",
    "repositoryId": "LorduyANDES/blacklist-microservice",
    "branchName": "main",
    "commitId": "5bd6ba064d5a5fd29fa3f2e0be4ca288263776fd",
    "type": "PUSH_BRANCH"
  }
}
```

Solo las dos ejecuciones más antiguas del historial **no** fueron automáticas: una es `CreatePipeline` (la que AWS lanza al crear el pipeline) y la otra `StartPipelineExecution` (lanzada a mano durante la construcción del pipeline, antes de que el disparador funcionara). La sección 8 explica por qué.

Conviene señalar que el historial sigue creciendo por sí solo. Los commits que publicaron esta misma documentación en el repositorio dispararon el pipeline sin ninguna intervención, y las pruebas se ejecutaron en todos ellos. No son ejecuciones preparadas para la entrega: son el comportamiento normal del repositorio desde que el disparador funciona.

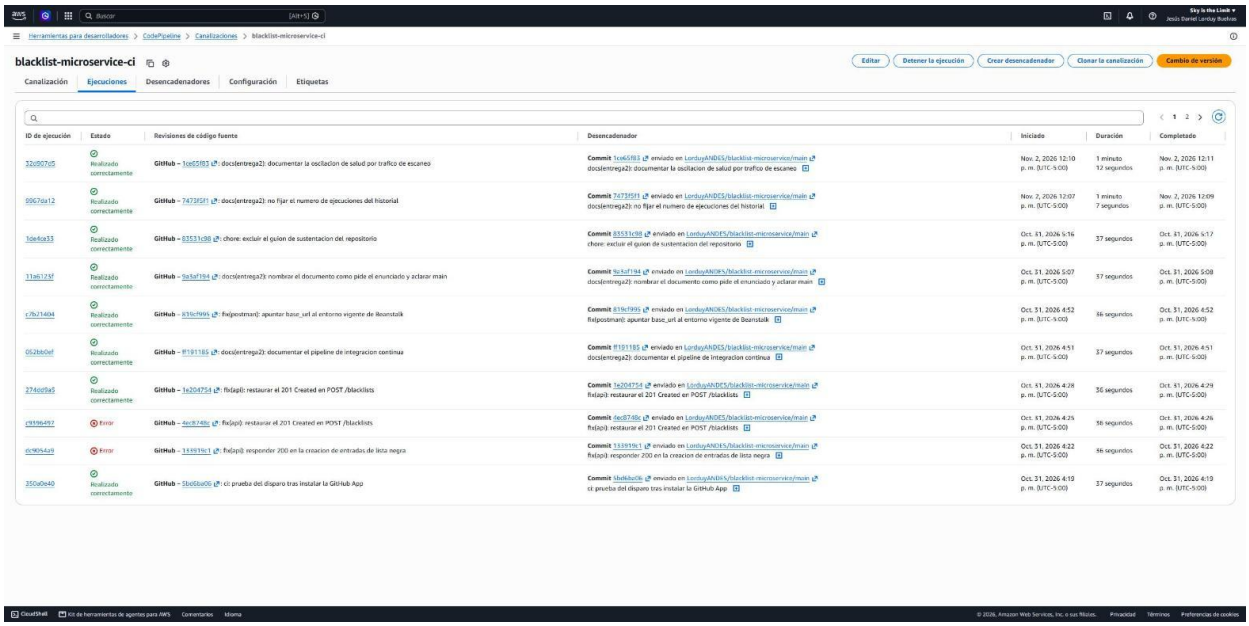


Figura 5. Historial de ejecuciones con la columna *Desencadenador*.

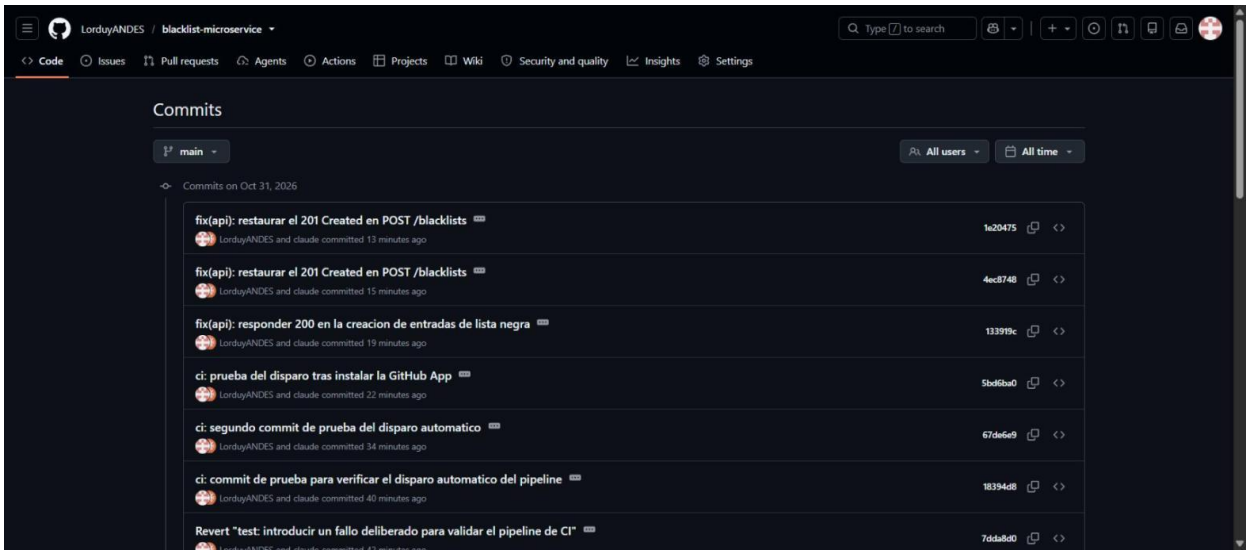


Figura 6. Commits de la rama `main` en GitHub que originaron esas ejecuciones.

---

7. Evidencia de una ejecución exitosa y una fallida

Para demostrar los dos caminos se introdujo deliberadamente una regresión y después se corrigió. Ambos cambios se publicaron como commits normales en `main`, de modo que **las dos ejecuciones se dispararon solas**.

La tabla siguiente recoge las cuatro ejecuciones de esa demostración. El historial completo contiene más, posteriores, correspondientes a los commits de documentación.

Ejecución	Commit	Mensaje	Resultado	Artefacto
`350a0e40`	`5bd6ba0`	`ci: prueba del disparo tras instalar la GitHub App`	Realizado correctamente	Sí
`dc9054a9`	`133919c`	`fix(api): responder 200 en la creacion de entradas de lista negra`	**Error**	No

Ejecución	Commit	Mensaje	Resultado	Artefacto
c9396497	4ec8748	fix(api): restaurar el 201 Created en POST /blacklists	**Error**	No
274dd9a5	1e20475	fix(api): restaurar el 201 Created en POST /blacklists	Realizado correctamente	Sí

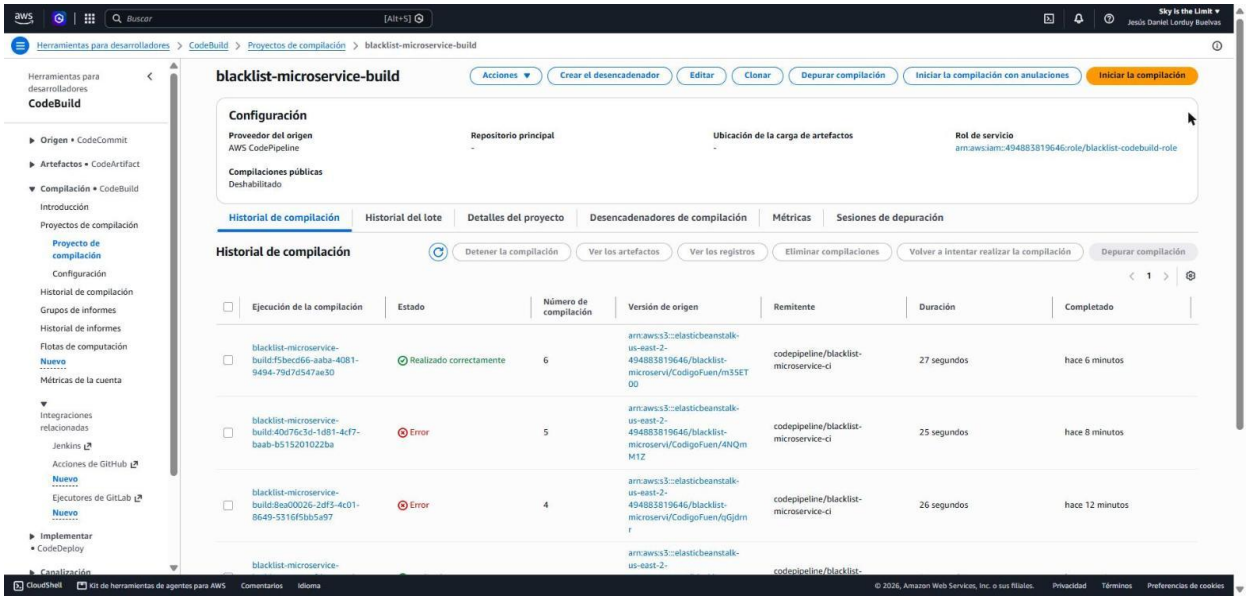


Figura 17. Historial de compilaciones de CodeBuild. La columna *Remitente* muestra `codepipeline/blacklist-microservice-ci` en todas, lo que confirma que las construcciones las originó el pipeline y no una ejecución manual de CodeBuild.

7.1 Ejecución exitosa

La construcción número 6 (`f5becd66`) corresponde al commit `1e20475`. Duró **27 segundos** y recorrió todas las fases:

```
SUBMITTED          SUCCEEDED
QUEUED             SUCCEEDED
PROVISIONING       SUCCEEDED   3 s
DOWNLOAD_SOURCE    SUCCEEDED   1 s
INSTALL            SUCCEEDED   0 s
PRE_BUILD          SUCCEEDED  18 s
BUILD              SUCCEEDED   0 s
POST_BUILD         SUCCEEDED   0 s
UPLOAD_ARTIFACTS   SUCCEEDED   0 s
FINALIZING         SUCCEEDED   0 s
```

El log muestra las 25 pruebas pasando:

```
collecting ... collected 25 items
tests/test_blacklists.py::test_health_check_responde_ok_sin_token PASSED  [  4%]
...
tests/test_blacklists.py::test_ruta_inexistente_responde_json PASSED      [100%]
============================== 25 passed in 0.66s ==============================
```

y a continuación el empaquetado del artefacto y su subida.

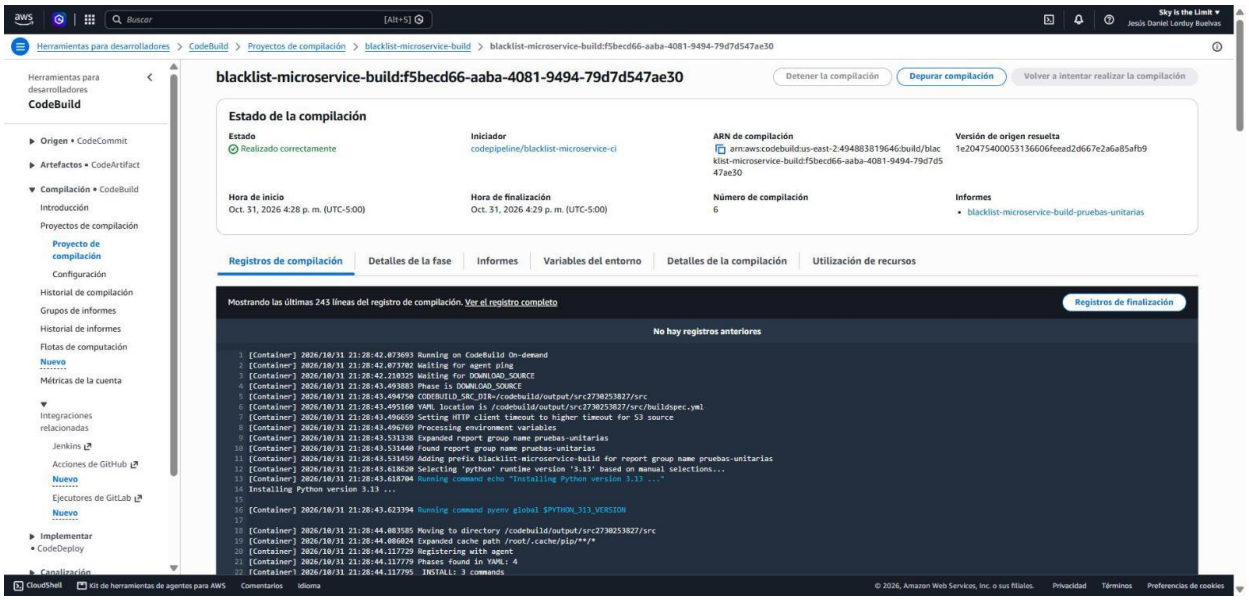


Figura 7. Estado de la construcción exitosa: *Realizado correctamente*, iniciador `codepipeline/blacklist-microservice-ci`, versión de origen `1e204754...`.

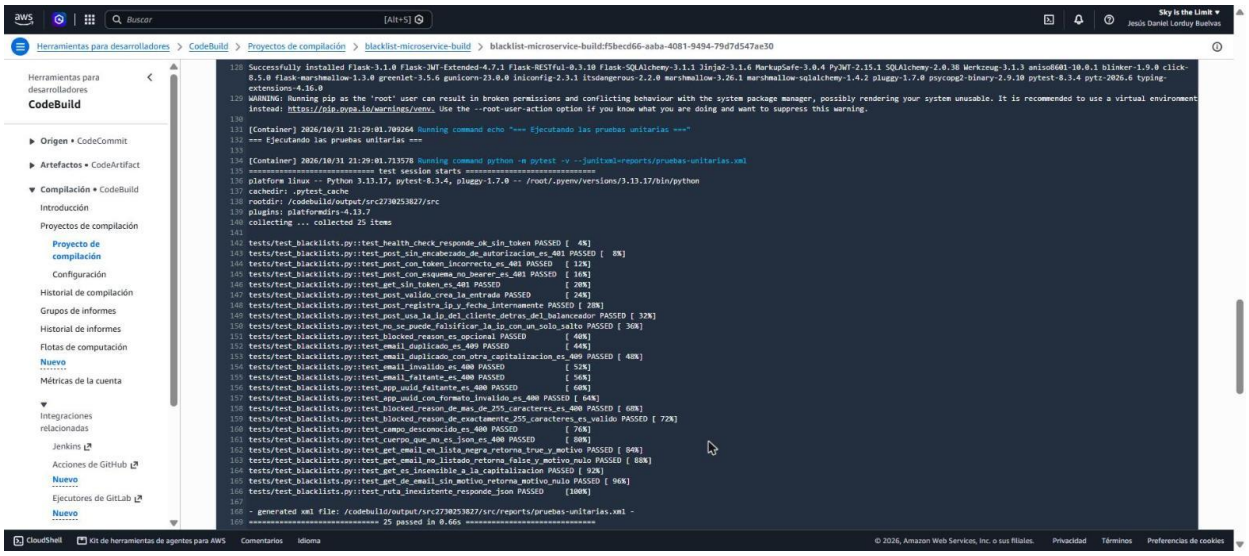


Figura 8. Log con las 25 pruebas en verde.

7.2 Ejecución fallida

Para provocar el fallo se rompió el contrato del endpoint de creación: `POST /blacklists` pasó a responder `200 OK` en lugar de `201 Created`. Es un cambio de una sola línea, plausible como descuido real, y que ninguna prueba de integración detectaría si no existieran las unitarias.

La construcción número 4 (`8ea00026`), correspondiente al commit `133919c`, falló en **`PRE_BUILD`**:

```
tests/test_blacklists.py::test_post_valido_crea_la_entrada FAILED          [ 24%]
tests/test_blacklists.py::test_blocked_reason_es_opcional FAILED           [ 40%]
tests/test_blacklists.py::test_blocked_reason_de_exactamente_255_caracteres_es_valido FAILED [ 72%]

================================== FAILURES ===================================
>       assert response.status_code == 201
E       assert 200 == 201
E         + where 200 = <WrapperTestResponse streamed [200 OK]>.status_code

======================== 3 failed, 22 passed in 0.76s =========================

[Container] Phase complete: PRE_BUILD State: FAILED
[Container] Phase context status code: COMMAND_EXECUTION_ERROR
            Message: Error while executing command:
```

```
        python -m pytest -v --junitxml=reports/pruebas-unitarias.xml.
        Reason: exit status 1
```

Las fases posteriores no se ejecutaron:

```
PRE_BUILD      FAILED    18 s
FINALIZING     SUCCEEDED  0 s
COMPLETED
```

No hubo `BUILD`, ni `POST_BUILD`, ni `UPLOAD_ARTIFACTS`. El artefacto nunca se generó y nada llegó a S3. Ese es exactamente el comportamiento buscado.

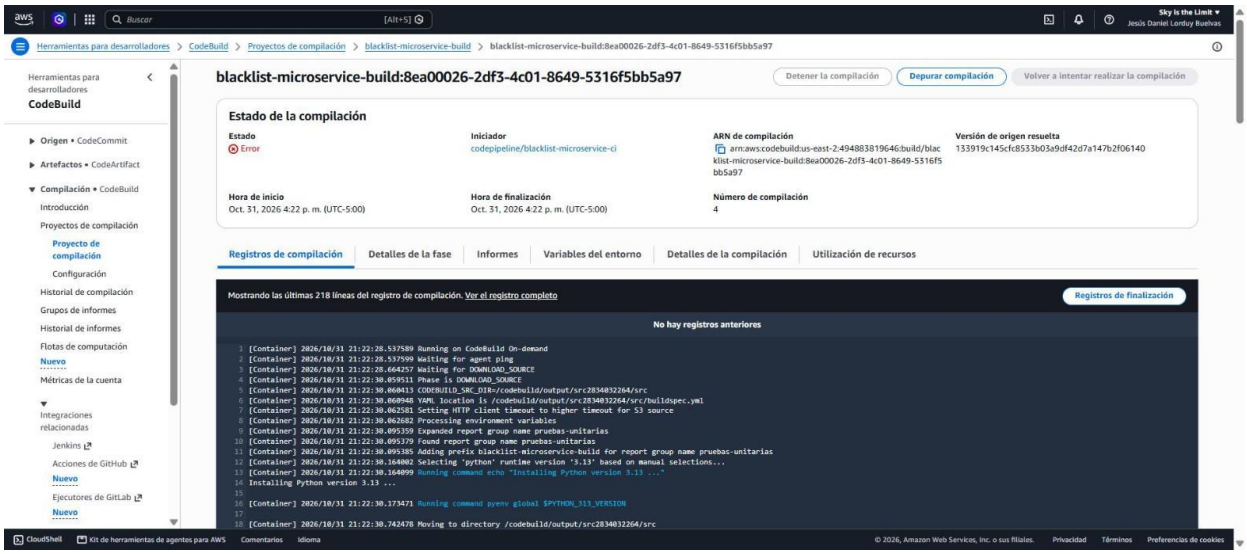


Figura 14. Estado de la construcción fallida: *Error*, versión de origen `133919c1...`.

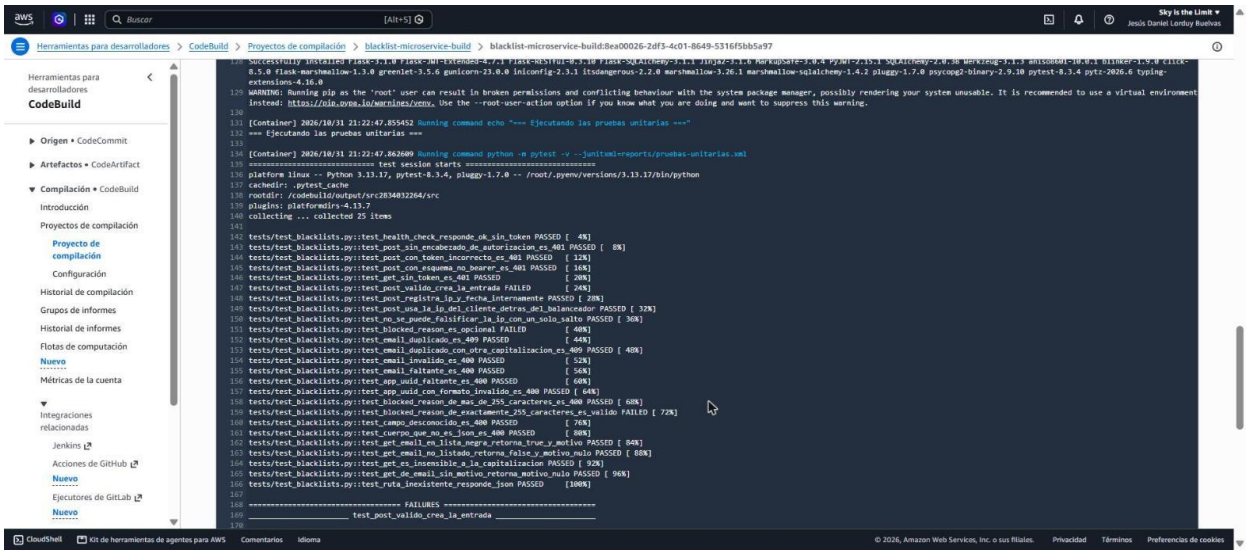


Figura 15. Log con las tres pruebas fallidas y el resumen `3 failed, 22 passed`.

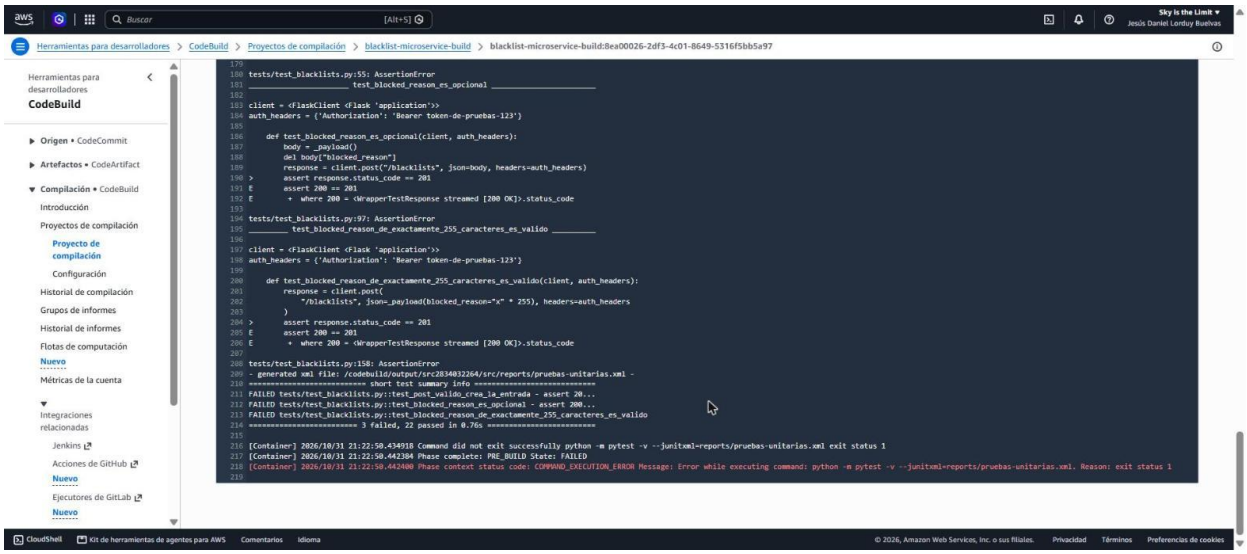


Figura 16. Detalle de las aserciones y fin de la fase `PRE_BUILD` en estado `FAILED`.

7.3 Nota sobre la tercera ejecución

La ejecución `c9396497` (commit `4ec8748`) también falló, y conviene explicarla en lugar de omitirla. Ese commit pretendía revertir la regresión, pero por un error al preparar el índice de Git solo registró la eliminación de un archivo temporal: la corrección de `views.py` se quedó sin publicar. El pipeline construyó el código tal como estaba en el repositorio —todavía con el `200`— y falló con las mismas tres pruebas. La corrección real llegó en `1e20475`, que sí pasó.

Es un fallo accidental, no planeado, y por eso resulta más ilustrativo que el fallo deliberado: el pipeline detectó que **lo que estaba en `main` no era lo que el autor creía haber subido**. Un pipeline de integración continua sirve justamente para eso.

---

8. Hallazgo: una conexión de GitHub «Disponible» puede no disparar el pipeline

Este fue el problema más costoso de la entrega, y su síntoma no apunta a su causa.

Tras crear el pipeline, la conexión de AWS CodeConnections aparecía en estado **Disponible**, la acción de origen leía el repositorio sin problema y la primera ejecución clonó correctamente el commit. Sin embargo, **ningún commit posterior disparaba el pipeline**. Las comprobaciones habituales no revelaban nada:

- `DetectChanges` estaba en `"true"`;
- el bloque `triggers` del pipeline V2 declaraba `push` sobre la rama `main`;
- el nombre de la acción de origen coincidía con `sourceActionName`;
- la rama por omisión del repositorio era efectivamente `main`;
- `aws codepipeline list-webhooks` devolvía una lista vacía;
- reaplicar la definición del pipeline no cambiaba nada.

La causa apareció en el registro de CloudTrail de la conexión:

```
15:23:47  CreateConnection
15:26:32  StartOAuthHandshake
15:27:01  GetIndividualAccessToken
15:29:44  UpdateConnectionOAuthToken
15:30:54  GetConnection  →  AVAILABLE
```

La secuencia completa la **autorización OAuth**, y nada más. Nunca se instaló la aplicación **AWS Connector for GitHub**. La página de aplicaciones instaladas de la cuenta de GitHub lo confirmaba: *«No installed GitHub Apps»*.

La distinción importa porque los dos mecanismos hacen cosas distintas:

Mecanismo	Para qué sirve
Autorización OAuth	Da a AWS un token con el que **leer** el repositorio
Instalación de la GitHub App	Es lo que hace que GitHub **entregue los eventos** de `push` a AWS

Con solo lo primero, AWS podía clonar el código cuando alguien lanzaba el pipeline a mano, pero no se enteraba nunca de un commit nuevo. De ahí el síntoma engañoso: una conexión «Disponible» que funciona para leer y no sirve para disparar.

Al instalar la aplicación sobre el repositorio, **la conexión original empezó a funcionar sin ningún otro cambio**: el siguiente commit disparó el pipeline con tipo `WebhookV2`.

Lección: el estado «Disponible» de una conexión de CodeConnections no garantiza que el disparo automático funcione. La comprobación concluyente es la lista de GitHub Apps instaladas en la cuenta propietaria del repositorio.

---

9. Por qué una construcción fallida no genera informe de pruebas

Una observación que conviene dejar registrada, porque podría interpretarse como un defecto de la configuración y no lo es.

Las construcciones exitosas publican su informe de pruebas en la pestaña *Informes*. Las fallidas **no**, pese a que pytest sí escribió el XML antes de terminar. El motivo está en el orden de fases: CodeBuild recoge y sube los informes durante `UPLOAD_ARTIFACTS`, y cuando `PRE_BUILD` falla la ejecución salta directamente a `FINALIZING`.

```
PRE_BUILD     FAILED
FINALIZING    SUCCEEDED
COMPLETED
```

Se evaluó mover las pruebas a una fase posterior para conseguir el informe también en los fallos, y se descartó: hacerlo significaría empaquetar el artefacto antes de saber si las pruebas pasan, que es exactamente lo que un pipeline de integración continua debe evitar. **Se prefirió la corrección del diseño sobre la comodidad de la evidencia.** El log de la construcción fallida documenta con total precisión qué pruebas fallaron y por qué, que es lo que se necesita para diagnosticar.

---

10. Estado de la aplicación

El microservicio permanece desplegado y operativo durante toda la entrega. El entorno `blacklist-env` está en estado **Ok**, sobre la plataforma Python 3.13 / Amazon Linux 2023, sirviendo la versión `e2-v1`.

```
GET http://blacklist-env.eba-2qryjvm9.us-east-2.elasticbeanstalk.com/
→ HTTP 200
```

La colección de Postman de la entrega 1 sigue siendo válida; basta actualizar la variable `base_url` del environment con la URL vigente.

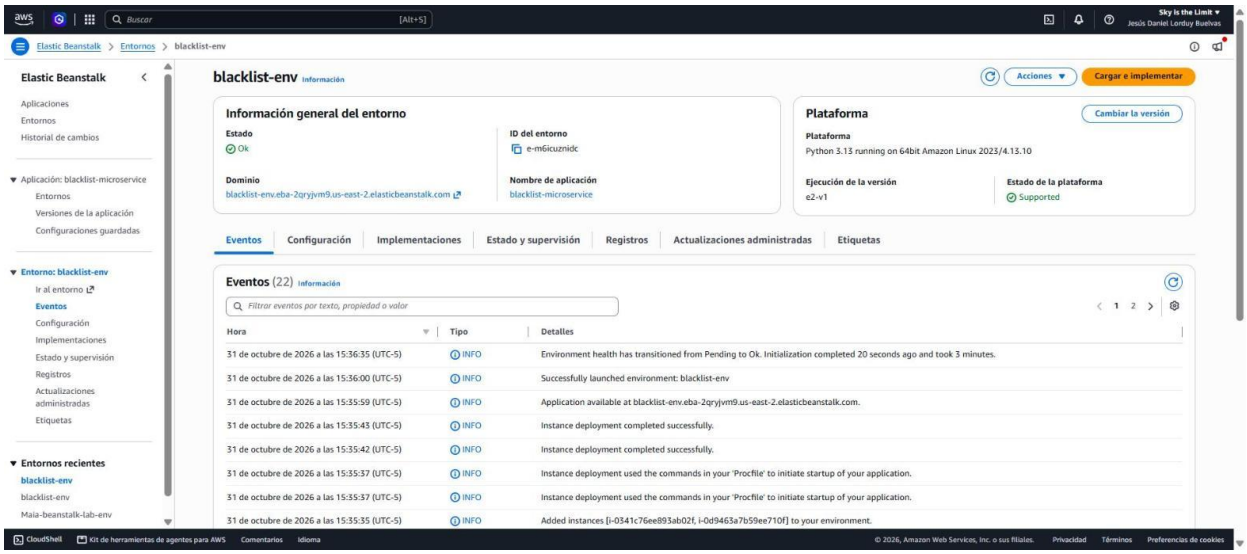


Figura 18. Entorno de Elastic Beanstalk en estado Ok.

10.1 Hallazgo: la salud del entorno oscila por tráfico de escaneo

El registro de eventos del entorno muestra transiciones de `Ok` a `Severe` y de vuelta a `Ok` en los días siguientes al despliegue, con mensajes del tipo:

```
Environment health has transitioned from Ok to Severe.
93.8 % of the requests are erroring with HTTP 4xx.
```

No corresponden a ningún fallo del servicio. La URL del entorno es pública, y recibe tráfico automatizado de escáneres de internet que prueban rutas comunes. El microservicio responde a ese tráfico exactamente como debe:

Petición	Respuesta
`GET /`	200
`GET /blacklists/<email>` sin token	**401**
`GET /wp-admin`, `GET /.env`, `GET /admin`	**404**

El modelo de salud mejorada de Elastic Beanstalk contabiliza las respuestas 4xx como errores de la aplicación, sin distinguir entre «la aplicación está rota» y «la aplicación está rechazando correctamente peticiones no autorizadas». Un microservicio cuyo contrato consiste precisamente en devolver 401 a quien no presenta un token válido va a parecer enfermo ante ese modelo cada vez que alguien lo escanee.

Lección: en un servicio autenticado, el porcentaje de 4xx no es un indicador útil de salud. El health check debe apuntar a una ruta pública y liviana —como hace este proyecto con `GET /`— y, en un entorno real, convendría ajustar las reglas de salud mejorada para que las respuestas 4xx no degraden el estado del entorno.

11. Conclusiones

3. **El pipeline cumple su función discriminante.** De las cuatro ejecuciones de la demostración, todas disparadas por commits, dos pasaron y dos fallaron, y las que fallaron no dejaron artefacto. La diferencia entre un pipeline que compila y uno que *verifica* está precisamente en que el segundo sabe decir que no.
4. **El orden de las fases es una decisión de diseño, no un detalle.** Poner las pruebas en `pre_build` convierte el empaquetado en una consecuencia de que las pruebas pasen, en vez de un paso paralelo e independiente.
5. **El fallo accidental del commit `4ec8748` resultó más valioso que el provocado.** Demostró el caso de uso real: no el de un desarrollador que rompe algo a propósito, sino el de uno que cree haber arreglado algo y no lo ha hecho.
6. **Un estado «Disponible» no es una verificación.** La conexión de GitHub se veía correcta en la consola y en la API mientras el disparo automático no funcionaba. La única comprobación concluyente fue observar un commit real disparando una ejecución real.
7. **Separar integración continua de entrega continua tiene un costo visible y deliberado.** El artefacto queda en S3 esperando un despliegue manual. Esa frontera es la que esta entrega debía respetar.

---

12. Declaración de uso de inteligencia artificial

Este proyecto se desarrolló con asistencia de un modelo de lenguaje (Claude, de Anthropic), empleado como herramienta de apoyo en la escritura de código, la configuración de los servicios de AWS, el diagnóstico de fallos y la redacción de este documento. Los commits del repositorio registran esa coautoría de forma explícita mediante la línea `Co-Authored-By`.

Todas las decisiones de arquitectura, las verificaciones sobre la consola de AWS y la validación de los resultados fueron revisadas por el autor. Las cifras, los estados y los logs reproducidos en este documento provienen de ejecuciones reales sobre la cuenta de AWS del proyecto y no fueron generados ni estimados por el modelo.

Anexo: cómo reproducir el pipeline

8. **Conexión a GitHub.** En la consola de AWS, *Herramientas para desarrolladores* → *Configuración* → *Conexiones* → *Crear conexión*, proveedor GitHub. Completar la autorización **y además instalar la aplicación «AWS Connector for GitHub»** sobre el repositorio. Sin este segundo paso la conexión queda «Disponible» pero no dispara nada (sección 8).
9. **Proyecto de CodeBuild.** Crear `blacklist-microservice-build` con origen *AWS CodePipeline*, imagen `amazonlinux-x86_64-standard:5.0`, cómputo EC2 y especificación de compilación tomada del archivo `buildspec.yml` del repositorio.

10. **Pipeline.** Crear `blacklist-microservice-ci` de tipo V2 con dos etapas: *Fuente* (acción `CodeStarSourceConnection` sobre la rama `main`, con `DetectChanges` activo) y *Construccion* (acción de CodeBuild, entrada `CodigoFuente`, salida `ArtefactoDesplegable`). **No añadir etapa de despliegue.**
11. **Verificación.** Registrar un commit en `main` y comprobar en la pestaña *Ejecuciones* que el desencadenador aparece como `Commit <sha> enviado en <repositorio>/main`. Si aparece `StartPipelineExecution`, el disparo no es automático.
12. **Prueba del camino de fallo.** Cambiar el código de respuesta de `POST /blacklists` de `201` a `200`, publicar el commit y confirmar que la construcción se detiene en `PRE_BUILD` y que no aparece artefacto nuevo en S3.